

❶ पहले समस्या को साफ समझते हैं (Single Graph Approach)

मान लो:

- 1 Billion vectors
- सबको एक ही **graph (single level)** में जोड़ दिया
- हर node अपने कुछ nearest neighbors से connected है

अब search कैसे हो रही है:

- Start node से search शुरू
- Min-heap / Max-heap से top-10 similar निकालने की कोशिश
- जैसे ही heap में 10 items आए → stop

✗ इसमें दिक्कत क्या है?

● Problem 1: Local Optima (Local Trap)

Graph में ऐसा हो सकता है:

- तुम एक **local dense area** में फँस गए
- वहाँ जल्दी 10 अच्छे neighbors मिल गए
- लेकिन **graph** के किसी और हिस्से में इससे भी ज़्यादा **similar vectors** मौजूद हैं

👉 Heap भर जाना ≠ Global best मिल जाना

● Problem 2: Search Path बहुत लंबा

1 Billion nodes में:

- हर hop = similarity calculation

- Graph flat है → jump करने का कोई shortcut नहीं
- Worst case में हजारों / लाखों nodes देखनी पड़ सकती हैं

👉 Time complexity explode हो जाती है

🔴 Problem 3: Graph में Direction नहीं है

Single level graph में:

- सब neighbors लगभग same “distance scale” पर हैं
 - Long-range jump possible नहीं
-

② अब HNSW का core idea समझो (Hierarchy)

HNSW बोलता है:

“सब nodes को एक ही graph में मत डालो
पहले ऊपर से नीचे धीरे-धीरे zoom करो 🔍”

③ HNSW Leveling क्या है?

HNSW में multiple **levels (layers)** होते हैं:

Level 3 (Very few nodes)	🌐	Global view
Level 2		
Level 1		
Level 0 (All nodes)	🔍	Detailed view

कैसे **levels** बनते हैं?

- Har node ko **random probability** से upper level assign होता है

- ऊपर के level पर बहुत कम **nodes** होते हैं

👉 Top level = sparse + long-range connections

👉 Bottom level = dense + accurate neighbors

4 Leveling क्यों ज़रूरी है? (Main Reason)

🎯 **Goal:**

Fast global navigation + accurate local search

5 Search कैसे होती है (Step-by-step)

♦ Step 1: Top Level (Global Jump)

- Search start होती है **highest level** से
- वहाँ बहुत कम nodes होते हैं (say 100–1000)
- यहाँ goal है:

“किस region में मेरा query vector सबसे पास है?”

👉 Large jumps possible

👉 Local trap नहीं

♦ Step 2: Level-by-Level Refinement

- Best node मिला → नीचे वाले level में उतरे
- हर level पर:
 - Search space छोटा होता जाता है

- Similarity ज्यादा accurate होती जाती है

◆ Step 3: Level 0 (Final Accurate Search)

- यहाँ **exact neighbors** खोजे जाते हैं
 - Heap यहाँ meaningful होती है
 - Top-K निकालना safe होता है
-

6 अब तुम वाले **Heap Approach** से तुलना

✗ Without Leveling (Single Graph)

Start → local cluster → heap fills → STOP ✗
(But better nodes exist elsewhere)

✓ With HNSW

Top level → correct region

↓

Mid level → closer region

↓

Bottom level → exact top-10

👉 Heap अब **correct area** में काम कर रही है

👉 Early stopping safe हो जाता है

7 Time Complexity क्यों सुधरती है?

Brute graph:

- $O(N)$ or $O(N \log K)$

HNSW:

- Approx:

$O(\log N)$ levels $\times$ small search per level
 $\approx O(\log N)$

For 1 Billion vectors:

- $\log_2(1B) \approx 30$ levels (practically 10–15)
-

8 Short Intuition Line (Interview / Notes)

HNSW leveling इसलिए करते हैं क्योंकि **single-level graph local optima** में फँस सकता है।

Hierarchy हमें पहले **globally** सही **region** ढूँढने देती है और फिर **locally accurate nearest neighbors** निकालने देती है — **fast** और **reliable** दोनों।

9 एक लाइन में conclusion

Leveling = GPS zoom system

- ऊपर से country देखो 🌐
- फिर city 🏙️
- फिर street 🛣️
- फिर exact house 🏠

HNSW में **layers (levels)** कैसे बनते हैं — इसे short + crystal clear तरीके से समझो, exactly implementation-level intuition के साथ 📌

1 Core idea (1 line)

HNSW layers random probability से बनती हैं, **distance** से नहीं।

यही चीज इसे fast + balanced बनाती है.

2 Layer assignment कब होता है?

👉 जब नया **vector insert** होता है, उसी time decide होता है:

- यह node कितने levels तक जाएगा
-

3 Random level generation (MOST IMPORTANT)

हर node के लिए एक **maxLevel** generate किया जाता है:

Formula (intuition level)

$$\text{level} = \lfloor -\ln(\text{rand}()) * \text{mL} \rfloor$$

Where:

- $\text{rand}()$ → uniform random number in (0,1]
- $\text{mL} = 1 / \ln(M)$
- M = max connections per node (usually 16)

👉 Result:

- ज्यादातर nodes → level 0
- कुछ nodes → level 1
- बहुत कम → level 2,3,4...

 Exponential decay distribution

4 Intuition: Probability कैसे घटती है?

मान लो:

Level	Probability
-------	-------------

0	$\sim 100\%$
---	--------------

1	$\sim 1/M$
---	------------

2	$\sim 1/M^2$
---	--------------

3	$\sim 1/M^3$
---	--------------

If $M = 16$:

- Level 1 $\rightarrow \sim 6\%$
- Level 2 $\rightarrow \sim 0.4\%$
- Level 3 $\rightarrow \sim 0.025\%$

👉 इसलिए top layers बहुत sparse होती हैं

5 Visual example

मान लो 1 million vectors हैं:

Level 3 $\rightarrow \sim 20$ nodes

Level 2 $\rightarrow \sim 300$ nodes

Level 1 $\rightarrow \sim 20,000$ nodes

Level 0 $\rightarrow 1,000,000$ nodes

↑ ऊपर global navigation

↓ नीचे accurate search

6 Node कैसे connect होता है layers में?

For each node:

For every level $l \leq \text{maxLevel}$:

1. उस level के graph में insert करो
2. Nearest neighbors खोजो (using search from upper level)
3. Top M neighbors से connect करो

👉 Upper levels:

- Long-range links
- Fewer neighbors

👉 Lower levels:

- Dense local connections

7 Important: Layer independent graphs हैं

- हर layer एक **separate graph** है
- Node सिर्फ उन layers में exist करता है जहाँ $l \leq \text{maxLevel}$

8 Why random levels? (Critical reason)

✗ Distance-based level assignment होता तो:

- Clusters skew हो जाते
- Graph unbalanced हो जाता

- ✓ Random = balanced + logarithmic height
(Same reason skip list works)
-

9 Short pseudocode

```
function getRandomLevel(M):  
    mL = 1 / ln(M)  
    return floor(-ln(random()) * mL)  
  
insert(node):  
    level = getRandomLevel(M)  
    for l = level downto 0:  
        search neighbors in layer l  
        connect node to top M neighbors
```

10 One-line summary (interview ready)

HNSW layers are created by assigning each node a random maximum level using an exponential distribution, ensuring sparse upper layers for fast global navigation and dense lower layers for accurate local search.

✓ ADVANTAGES of HNSW

1 Extremely fast search (sub-linear)

- Single graph में search = $O(N)$
- HNSW में:

$O(\log N)$

- Billion-scale vectors पर भी milliseconds में result

👉 Top reason why vector DBs use HNSW

2 Avoids local optima trap

- Leveling की वजह से:
 - पहले global region identify
 - फिर local refinement

✗ Flat graph → local cluster में फँस सकता है
✓ HNSW → correct region तक jump

3 Very high recall (near-exact)

- Recall 95–99% possible
- IVF / PQ से ज़्यादा accurate

👉 Search speed + accuracy का best balance

4 Online / incremental insertion

- Data streaming में add कर सकते हो
- Re-index की ज़रूरत नहीं

👉 Real-time systems के लिए perfect

5 Distance-agnostic

- Any metric:
 - Cosine

- L2
- Inner product

👉 Embeddings friendly

6 Self-balancing structure

- Random level assignment
- Skip list-like behavior

👉 No manual tree balancing

7 Simple concept, powerful result

- Graph + hierarchy
 - No heavy math like PQ
-

✗ DISADVANTAGES of HNSW

1 High memory usage !

- Har node:
 - Multiple layers
 - Multiple neighbors

Memory $\approx$ **10–20× raw vectors**

👉 Biggest drawback

2 Build time expensive

- Insertion = multiple searches
- Graph connections expensive

👉 Index build slow for very large datasets

3 No exact guarantee

- Approximate NN (ANN)
- 100% exact nearest neighbor not guaranteed

👉 Accuracy $\neq$ brute force

4 Delete/update is hard

- Node delete करना complex
- Graph consistency issue

👉 Most DBs:

- Mark as deleted
 - Rebuild index later
-

5 Parameter sensitive

Important parameters:

- $M \rightarrow$ neighbors per node

- efConstruction
- efSearch

✗ Wrong tuning → slow or low recall
✓ Correct tuning → amazing performance

6 Not disk-friendly by default

- Random memory access
- RAM-heavy

👉 Disk-based systems में IVF बेहतर होता है

Quick Comparison

Feature	HNSW	Brute Force	IVF
Speed	★★★★★	✗	★★★★
Recall	★★★★★	★★★★★	★★★★
Memory	✗	★	★★★★
Online insert	✓	✗	✗
Complexity	Medium	Simple	Complex

चलो **Blinkit-style recommendation** को **step-by-step, system design + ML intuition** के साथ समझते हैं — exactly “milk add to cart” के बाद क्या होता है।

Scenario

Deepak ने Blinkit पर “milk” search किया
20 प्रकार के milk दिखे
Deepak ने एक milk add to cart किया

अब सवाल 🙌 अब Blinkit क्या recommend करेगा और क्यों?

❶ सबसे पहले समझो: ये Search नहीं है, Recommendation है

Search:

- User query = “milk”

Recommendation:

- User action = add to cart (intent signal)

🙌 Add to cart = strong intent signal 🔥

❷ Recommendation के 3 main signals

♦ A) Item-to-Item Similarity (MOST IMPORTANT)

Blinkit internally पूछता है:

“इस milk को add करने वाले users ने और क्या add किया?”

Examples:

- Same brand, different pack
- Same category (curd, paneer, butter)
- Frequently bought together

📌 This is collaborative filtering

♦ B) Contextual Rules (Business logic)

Milk add हुआ → system सोचता है:

Milk → Morning / Daily essentials

So rules trigger:

- Bread
- Eggs
- Butter
- Cereal
- Tea / Coffee
- Sugar

👉 This is **rule-based + ML hybrid**

♦ C) User Personalization (Deepak specific)

Blinkit checks Deepak's history:

- You usually buy:
 - Amul?
 - Toned milk?
 - Low-fat?
- You buy eggs or not?
- Veg / Non-veg pattern

👉 Ranking personalized हो जाती है

③ Internally क्या होता है (Tech flow)

Step 1: Event fired

Event: ADD_TO_CART

User: Deepak

Item: Amul Toned Milk 500ml

Step 2: Candidate generation (fast)

Candidates आते हैं:

1. Same category items
2. Bought-together items
3. Substitute items
4. Promotions

~100–500 items

Step 3: Vector similarity (HNSW plays here)

हर item का embedding होता है:

- Category
- Brand
- Nutrition
- Price
- User behavior

Query vector = "Milk + Deepak context"

➡ HNSW top-K similar items निकालता है

Step 4: Ranking model

Final rank based on:

- Similarity score
 - Past purchases
 - Time of day
 - Stock availability
 - Margin (business 💰)
-

4 What Blinkit will ACTUALLY show (Examples)

Immediate milk-related

- Same milk (1L pack)
 - Same brand low-fat version
 - Combo offer milk + curd
-

Common co-purchases

- Bread
- Eggs
- Butter

- Paneer
 - Curd
-

Smart suggestions

- Cereal (if morning time)
 - Tea / Coffee (if frequent buyer)
 - Discounted essentials
-

5 UI placements

Section	Example
“Frequently bought together”	Milk + Bread
“You may also like”	Curd, Paneer
“Upgrade your pack”	1L Milk
“Deals for you”	Combo offers

6 Why not random items?

-  No shampoo
-  No mobile charger

Because:

- Add to cart narrows intent
 - Recommendation shifts from **explore** → **complete basket**
-

7 One-line system thinking

Search finds what you ask for.

Add-to-cart reveals what you're about to consume — and recommendation completes your basket.

Interview-ready summary

When a user adds milk to cart, Blinkit uses item-to-item similarity, co-purchase signals, and user personalization to recommend complementary and substitute daily essentials, ranked using embedding similarity (often via HNSW) and business constraints.

PRODUCT QUANTIZATION

1 पहले reality check (numbers कभी झूठ नहीं बोलते)

Single vector

- Dimension = 1536
- Data type = float32 (4 bytes)

$$1536 \times 4 = 6,144 \text{ bytes} \approx 6 \text{ KB}$$

1 million vectors

$$6 \text{ KB} \times 1,000,000 \approx 6 \text{ GB}$$

Important correction:

1 million = 6 GB

1 billion = ~6 TB

 Production systems usually have 100M–1B vectors

2 अब असली problem क्या है?

मान लो:

- 500 million products / embeddings
- Each = 6 KB

Memory:

500M × 6 KB ≈ 3 TB ❌

अब सोचो:

- RAM में impossible
- Disk से read = slow
- Cache thrash
- Cost skyrocket 💰

👉 Exact vectors रखना scalable नहीं है

3 Search का hidden cost (bigger problem)

Even if memory somehow available:

For 1 query:

Distance calc = 1536 multiplications
× millions of vectors

- 👉 CPU/GPU burn
 - 👉 Latency increase
-

4) यही pain point से PQ आया 🔥

"Do we really need exact 1536 floats for similarity?"

Answer: ❌ NO

Similarity search:

- Slight error acceptable
- Rank matters, not exact distance

👉 Approximation allowed

5) Product Quantization = smart compromise

Core idea (plain English):

Vector को छोटे parts में तोड़ दो

Har part को "closest prototype" से represent कर दो

6) Numbers with PQ (eye-opener)

Assume:

- 1536 dims
- Split into $M = 96$ sub-vectors
- Each sub-vector $\rightarrow$ 1 byte code

Memory per vector:

96 bytes

Compare:

Original: 6,144 bytes

PQ: 96 bytes

🔥 64× compression

7 Million / Billion scale with PQ

1 million vectors

96 MB ✓

1 billion vectors

96 GB ✓ (possible on machines / disk)

👉 Suddenly billion-scale possible

8 Search कैसे fast रहती है?

Key trick:

Asymmetric Distance Computation (ADC)

- Query = full precision
- Stored vectors = compressed
- Precomputed lookup tables

👉 No decompression

👉 Fast distance approx

9 Why not just reduce dimension?

✗ PCA alone:

- Loses semantic meaning

✗ Float16:

- Only 2× saving

✓ PQ:

- Huge compression
- Minimal quality loss

10 Mental model (remember forever)

PQ = JPEG for vectors

Image exact नहीं रहती

But perception almost same

🎯 Final one-line answer (interview gold)

Because storing and searching billions of 1536-dimensional float vectors is prohibitively expensive in memory and computation, Product Quantization was introduced to compress vectors by orders of magnitude while preserving approximate similarity for fast large-scale retrieval.

अब हम Product Quantization (PQ) को numbers के साथ, step-by-step exactly तुम्हारे दिए हुए vector पर समझते हैं।
कोई abstraction नहीं — pure working.

📌 Given

Original Vector (dimension = 16)

$V = [$

```
1.1, 2.3, 0.9, 3.4,  
8.8, 7.6, 9.1, 6.5,  
4.2, 5.5, 3.9, 5.1,  
9.9, 8.1, 7.7, 9.3  
]
```

Storage (float32)

$16 \times 4 \text{ bytes} = 64 \text{ bytes}$

Step-1: Decide PQ parameters

For simplicity:

- Dimension = 16
 - Choose $M = 4$ sub-vectors
 - Each sub-vector dimension = $16 / 4 = 4$
 - Codebook size per sub-vector = $K = 256$ (1 byte)
-

Step-2: Split vector into sub-vectors

```
V1 = [1.1, 2.3, 0.9, 3.4]  
V2 = [8.8, 7.6, 9.1, 6.5]  
V3 = [4.2, 5.5, 3.9, 5.1]  
V4 = [9.9, 8.1, 7.7, 9.3]
```

👉 This is “Product” in Product Quantization
(full vector = product of subspaces)



Step-3: Train codebooks (offline)

For each sub-vector position (V1, V2, V3, V4):

- Collect that part from all vectors
- Run K-means (K=256)

Example (for V1 subspace):

Centroid 12 → [1.0, 2.1, 1.0, 3.5]

Centroid 87 → [0.8, 2.4, 0.7, 3.2]

...

📌 Same for V2, V3, V4

📌 Total codebooks = 4



Step-4: Encode the vector (compression)

Now each sub-vector is replaced by nearest centroid ID.

Example mapping:

Sub-vector	Nearest centroid	Stored
V1	ID 12	12
V2	ID 201	201
V3	ID 67	67
V4	ID 145	145



Final PQ-encoded vector

$PQ(V) = [12, 201, 67, 145]$

Storage:

$4 \text{ sub-vectors} \times 1 \text{ byte} = 4 \text{ bytes}$



From 64 bytes $\rightarrow$ 4 bytes

$16\times$ compression



Step-5: Similarity search (MOST IMPORTANT)

Query vector (Q) remains FULL precision:

$Q = [\dots 16 \text{ floats} \dots]$

Asymmetric Distance Computation (ADC)

1. Split query same way:

Q_1, Q_2, Q_3, Q_4

2. For each subspace:

Compute distance:

$Q_1 \leftrightarrow \text{all } 256 \text{ centroids}$

$Q_2 \leftrightarrow \text{all } 256 \text{ centroids}$

$\dots$

- Store in lookup tables

3. Distance to PQ vector:

```
dist(Q, V) =  
    table1[12] +  
    table2[201] +  
    table3[67] +  
    table4[145]
```

- 👉 No vector reconstruction
 - 👉 Just table lookup + sum
-

Intuition lock (remember forever)

- ✗ We don't store real numbers
- ✓ We store addresses to typical patterns

Like:

“This part looks like pattern #12”

Trade-off

Aspect	Value
Memory	🚀 Huge saving
Speed	⚡ Very fast
Accuracy	⚠ Approximate
Exactness	✗ Lost



Final 1-line summary

Product Quantization compresses a 16-dimensional vector by splitting it into sub-vectors, replacing each with a centroid ID, reducing storage from 64 bytes to 4 bytes while enabling fast approximate similarity search via lookup tables.